

POC MQTT — Communication Machines Industrielles vers MES

POC réalisé dans le cadre de mon stage chez **Capgemini Technology Services** (Issy-les-Moulineaux, juillet–août 2025) au sein de l'équipe **SaaS Platform**, sous l'encadrement de **Mouhamed DIOUF** (Lead SaaS Platform Architect).

Contexte

Dans une usine moderne, des dizaines voire centaines de machines doivent remonter en temps réel leur état (température, vitesse, alarmes) à un système central appelé **MES** (*Manufacturing Execution System*). Le MES pilote la production, suit les ordres de fabrication, collecte les données qualité.

Problématique : comment faire communiquer des machines hétérogènes (constructeurs, langages et OS différents) avec le MES de manière fiable, asynchrone et scalable ?

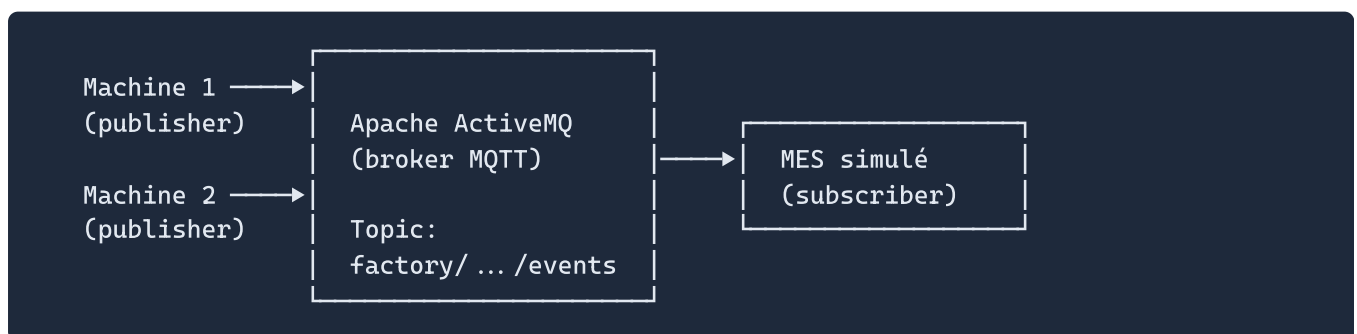
Réponse : utiliser une **architecture événementielle** basée sur un **broker de messages**. Chaque machine publie ses événements sur des topics MQTT, et le MES s'y abonne pour recevoir ce qui l'intéresse, sans couplage fort entre les composants.

Objectif du POC

Démontrer la faisabilité d'un système de messaging asynchrone entre des machines (simulées) et un consommateur (qui simule un MES), en utilisant :

- **Apache ActiveMQ** comme broker multi-protocoles
- **MQTT** comme protocole côté machines (léger, orienté IoT)
- **Python** pour les clients publisher / subscriber
- **Docker Compose** pour reproduire l'environnement en une commande

Architecture



Stack technique

Composant	Technologie	Rôle
Broker	Apache ActiveMQ 6.1	Routage des messages
Protocole	MQTT 3.1.1	Communication client ↔ broker
Langage	Python 3.11+	Clients publisher / subscriber
Bibliothèque	paho-mqtt 2.1	Implémentation MQTT en Python
Conteneurisation	Docker / Docker Compose	Lancement reproductible

Installation et démarrage

Prérequis

- Docker + Docker Compose
- Python 3.11 ou plus récent

1. Lancer le broker ActiveMQ

```
docker-compose up -d
```

Console web disponible sur <http://localhost:8161> (login : `admin` / `admin`)

2. Installer les dépendances Python

```
pip install -r requirements.txt
```

3. Lancer le subscriber (MES simulé)

Dans un terminal :

```
python subscriber.py
```

4. Lancer le publisher (machine simulée)

Dans un autre terminal :

```
python publisher.py
```

Tu verras les événements machine apparaître côté subscriber en temps réel.

5. Arrêter

```
docker-compose down
```

Configuration

Les deux scripts acceptent des variables d'environnement :

Variable	Défaut	Description
MQTT_BROKER	localhost	Adresse du broker
MQTT_PORT	1883	Port MQTT
MQTT_TOPIC	factory/machine1/events	Topic publié / écouté
NB_MESSAGES	5	Nombre de messages (pub)
DELAY	1.0	Délai entre messages (s)

Exemple :

```
MQTT_TOPIC=factory/machine2/events NB_MESSAGES=20 python publisher.py
```

Concepts clés expliqués

Architecture événementielle (Event-Driven Architecture)

Au lieu que les composants s'appellent directement (request/response), ils communiquent via des **événements** publiés sur un broker. Avantages : **découplage**, **scalabilité**, **résilience** (les événements sont stockés si un consommateur est temporairement indisponible).

Message broker

Un **broker de messages** est un middleware qui reçoit les messages des producteurs et les distribue aux consommateurs. Analogie : c'est un bureau de poste — tu n'as pas besoin de connaître l'adresse exacte du destinataire, le facteur s'en charge.

MQTT (Message Queuing Telemetry Transport)

Protocole **léger** créé en 1999 par IBM pour la télémétrie satellite. Aujourd'hui omniprésent dans l'IoT (capteurs, voitures connectées, domotique). Trois caractéristiques : - **Lightweight** : faible empreinte réseau et CPU - **Pub/Sub** : producteurs et consommateurs ne se connaissent pas - **QoS** : 3 niveaux de qualité de service (0 = au plus une fois, 1 = au moins une fois, 2 = exactement une fois)

ActiveMQ

Broker open source d'Apache. Multi-protocole : il parle MQTT, AMQP, STOMP, OpenWire (JMS). Permet l'**interopérabilité** : un message MQTT peut être consommé par un client JMS (utile pour faire dialoguer des systèmes legacy avec de l'IoT moderne).

Apprentissages personnels

Ce POC m'a permis de : - Comprendre les principes des architectures distribuées événementielles - Déployer et configurer un broker en environnement de développement - Manipuler la conteneurisation Docker pour reproduire des environnements - Programmer un système publisher/subscriber asynchrone en Python - Appréhender les enjeux de l'IoT industriel et du MES

Pistes d'amélioration

- Ajouter de l'authentification TLS sur MQTT
- Persister les événements dans une base time-series (InfluxDB)
- Ajouter un dashboard temps réel avec Grafana
- Conteneuriser également les clients Python
- Ajouter des tests unitaires (pytest)

Auteur

Mouhammad El Bachir Dieng — BTS SIO SLAM — IRIS Paris